

Министерство науки и высшего образования Российской Федерации  
Московский физико-технический институт (государственный университет)

Физтех-школа прикладной математики и информатики  
Кафедра системного программирования ИСП РАН  
Лаборатория (laboratory name)

Выпускная квалификационная работа бакалавра

Применение многопоточного кольцевого буфера для  
организации высокоскоростного обмена  
диагностической информацией в среде виртуализации

**Автор:**

Студент Б05-232 группы  
Абубакиров Владимир Андреевич

**Научный руководитель:**

Кандидат физико-математических наук  
Камкин Александр Сергеевич

**Научный консультант:**

\*научная степень\*

Скородумов Дмитрий Павлович



Москва 2026

### **Аннотация**

Применение многопоточного кольцевого буфера для организации высокоскоростного обмена диагностической информацией в среде виртуализации

*Абубакиров Владимир Андреевич*

Современные платформы виртуализации реализуют сложный набор подсистем, обеспечивающих взаимодействие между гостевыми и хостовыми операционными системами. Однако для задач низкоуровневой отладки, профилирования и формирования корректных отчётов об ошибках критически важным является наличие высокоэффективного канала передачи диагностических данных из гостевой операционной системы на хост. В рамках работы был разработан подобный механизм, основанный на lock-free кольцевом буфере в общей памяти.

## Содержание

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>Введение</b>                                              | <b>5</b>  |
| <b>2</b> | <b>Архитектура виртуальной машины Stratovirt</b>             | <b>11</b> |
| 2.1      | Stratovirt в экосистеме виртуализации . . . . .              | 11        |
| 2.2      | Архитектура Stratovirt . . . . .                             | 11        |
| 2.3      | Модель устройств . . . . .                                   | 12        |
| 2.4      | Добавление LogDev через параметры командной строки . . . . . | 13        |
| 2.5      | Регистровый интерфейс LogDev (MMIO) . . . . .                | 13        |
| 2.6      | ACPI-описание для гостевых систем . . . . .                  | 14        |
| 2.7      | Обработка диагностического сообщения в эмуляторе . . . . .   | 14        |
| 2.8      | Сравнение с альтернативными каналами . . . . .               | 15        |
| 2.9      | Выводы по главе . . . . .                                    | 15        |
| <b>3</b> | <b>Драйверы в среде UEFI</b>                                 | <b>17</b> |
| 3.1      | Обзор фаз Platform Initialization . . . . .                  | 17        |
| 3.2      | Фаза SEC (Security) . . . . .                                | 18        |
| 3.3      | Фаза PEI (Pre-EFI Initialization) . . . . .                  | 19        |
| 3.3.1    | PEI до инициализации ОЗУ (pre-memory) . . . . .              | 19        |
| 3.3.2    | PEI после инициализации ОЗУ (post-memory) . . . . .          | 19        |
| 3.4      | Фаза DXE (Driver Execution Environment) . . . . .            | 20        |
| 3.5      | Фаза TSL (Transient System Load) . . . . .                   | 21        |
| 3.6      | Фаза Runtime . . . . .                                       | 21        |
| 3.7      | Перезагрузка . . . . .                                       | 22        |
| 3.8      | Отладочная библиотека . . . . .                              | 22        |
| 3.9      | Выводы по главе . . . . .                                    | 22        |
| <b>4</b> | <b>Драйверы в операционной системе Windows</b>               | <b>24</b> |
| 4.1      | Задача драйвера устройства логирования . . . . .             | 24        |
| 4.2      | Схема взаимодействия с драйвером . . . . .                   | 24        |
| 4.3      | Модель разрабатываемого драйвера . . . . .                   | 25        |
| 4.4      | Обнаружение буфера через ACPI . . . . .                      | 25        |
| 4.5      | Интерфейс IOCTL для пользовательского режима . . . . .       | 26        |
| 4.6      | Интерфейс для драйверов режима ядра . . . . .                | 26        |

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| 4.7      | Взаимодействие с Stratovirt после записи . . . . .           | 27        |
| 4.8      | Сравнение путей доступа . . . . .                            | 27        |
| 4.9      | Выводы по главе . . . . .                                    | 27        |
| <b>5</b> | <b>Реализация механизма сбора диагностической информации</b> | <b>29</b> |
| <b>6</b> | <b>Заключение</b>                                            | <b>38</b> |

## 1 Введение

Развитие информационной инфраструктуры, начиная от облачных платформ и центров обработки данных, заканчивая персональными компьютерами, всё чаще полагается на технологии виртуализации.

Виртуализация — группа технологий, основанных на преобразовании формата или параметров программных или сетевых запросов к компьютерным ресурсам с целью обеспечения независимости процессов обработки информации от программной или аппаратной платформы информационной системы.<sup>1</sup>

Простые пользователи используют т.н. виртуальные машины для запуска альтернативных операционных систем на своих рабочих станциях для тестирования подозрительных программ, для разработки и обучения. Облачные провайдеры используют виртуальные машины для обеспечения работы серверных платформ: в частности, для предоставления пользователям возможности аренды вычислительных ресурсов. Последним это позволяет не покупать собственное оборудование для разворачивания инфраструктуры, масштабировать мощности под нагрузку и платить только за фактическое использование ресурсов.

Виртуальная машина — виртуальная вычислительная система, которая состоит из виртуальных устройств обработки, хранения и передачи данных и которая дополнительно может содержать программное обеспечение и пользовательские данные.<sup>2</sup>

Отдельным фактором, влияющим на рост спроса на технологии виртуализации, является развитие машинного обучения и искусственного интеллекта. Развертывание и обучение моделей требует больших вычислительных ресурсов (мощных GPU и NPU). Облачные провайдеры предоставляют виртуальные машины с таким оборудованием, позволяя бизнесу и частным поль-

---

<sup>1</sup>ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

<sup>2</sup>ГОСТ Р 56938-2016. Национальный стандарт Российской Федерации. Защита информации. Защита информации при использовании технологий виртуализации. Общие положения

зователям проводить исследования, не приобретая дорогостоящее оборудование. Это дополнительно увеличивает потребность эффективного использования аппаратного обеспечения, что также достигается при помощи технологий виртуализации.

Развитие технологий виртуализации предъявляет повышенные требования к инструментам диагностики и отладки, которые должны обеспечивать сбор данных из гостевых операционных систем с минимальным влиянием на их временные характеристики. В задачах логгирования и профилирования ключевыми являются низкая задержка и высокая пропускная способность передачи диагностической информации. В связи с этим возникает необходимость создания специализированного канала связи «гость – хост», оптимизированного для однонаправленной потоковой передачи данных.

Ситуация усугубляется тем, что на одной физической рабочей станции может одновременно функционировать несколько виртуальных машин. Каждая из них генерирует свои собственные логи, операции записи которых конкурируют за общие аппаратные ресурсы. Таким образом, механизм сбора диагностических данных должен работать с минимальной задержкой не только для отдельно взятой виртуальной машины, но и масштабироваться при одновременной работе множества экземпляров.

Ещё одной особенностью современных виртуальных машин является наличие сопутствующего программного обеспечения, позволяющего пользователям бесшовно с ними взаимодействовать. Например, инструменты, обеспечивающие общий буфер обмена, перенаправление USB-устройств, синхронизацию времени и разрешения экрана. Эти системы в процессе своей работы также генерируют диагностические события, которые могут быть полезны для выявления ошибок у пользователей. Таким образом, система сбора диагностической информации должна эффективно работать с учётом функционирования нескольких самостоятельных компонентов внутри виртуальной машины.

Разработка подобной системы передачи диагностических данных под-

разумеает создание драйвера в гостевой операционной системе, который взаимодействует со специальным виртуальным устройством на стороне эмулятора. Данное устройство выступает в роле конечного приёмника сообщений, обеспечивая их сбор и последующую печать в файл. Такой подход позволяет сохранить прозрачность для пользователя, так как гостевые операционные системы воспринимают устройство как стандартное аппаратное обеспечение.

Основой данного механизма является общая память, разделяемая между гостевой операционной системой и гипервизором. Такой подход позволяет организовать канал передачи данных без применения медленных операций ввода-вывода и без дополнительных копирований между буферами.

Сложность разработки заключается в том, что в угоду минимальных задержек, синхронизация доступа к буферу осуществляется без блокировок, благодаря использованию атомарных операций, что исключает риски взаимных блокировок и обеспечивает более детерминированное время выполнения операций. Такой подход позволяет достичь высокой пропускной способности и минимальной задержки при операциях записи, что особенно важно для работы с несколькими виртуальными машинами.

Немаловажной задачей также является реализовать механизм передачи сообщений из среды BIOS (UEFI). На этом этапе происходит первичная инициализация аппаратных компонентов и сбои приводят к невозможности дальнейшей загрузки системы. Отсутствие средств сбора отладочной информации существенно затрудняет выявление причин отказов.

BIOS представляет собой интерфейс между программным обеспечением и аппаратными средствами, который позволяет им общаться и взаимодействовать друг с другом.<sup>3</sup>

UEFI (от англ. Unified Extensible Firmware Interface, единый расширяемый микропрограммный интерфейс) был разработан консорциумом UEFI Forum на основе спецификаций, изначально предоставленных корпорацией Intel как преемника BIOS (хотя большинство образов микропрограммного обеспечения UEFI предоставляют поддержку для устаревших сервисов BIOS).

---

<sup>3</sup>Вонг А. Справочник по параметрам BIOS. — 2005

UEFI обеспечивает поддержку современного оборудования, модульной архитектуры, не зависящей от архитектуры ЦПУ и драйверов, загрузочной сети и безопасной загрузки.<sup>4</sup>

Важным преимуществом предлагаемого решения является возможность вывода логов от гостевой операционной системы вместе с диагностическими данными от самого эмулятора. В результате логи драйверов и сервисов, работающих в гостевых операционной системах, и сообщения эмулятора печатаются в одном месте в хронологическом порядке. Это позволяет видеть полную картину и более эффективно выявлять и исправлять ошибки. Техподдержке и разработчикам не нужно собирать диагностическую информацию из разных источников и пытаться сопоставить события по времени, ведь в одном файле доступны все логи, упорядоченные по времени.

Для решения проблем, описанных выше, предлагается механизм, основанный на использовании общей памяти. На стороне эмулятора реализуется виртуальное устройство, служащее интерфейсом для хранения метаданных о буфере. В операционной системе разработан драйвер, обеспечивающий взаимодействие с данным устройством и передачу диагностических данных.

Объект исследования в данной работе — высокопроизводительный механизм передачи информации между гостевой операционной системой и эмулятором с минимальными временными задержками.

Предмет исследования — многопоточный кольцевой буфер без блокировок как средство передачи диагностической информации в среде виртуализации.

Целью дипломной работы является разработка комплекса программных компонентов для организации высокоскоростной передачи диагностической информации в среде виртуализации. В состав разрабатываемого решения входят: виртуальное устройство для виртуальной машины Stratovirt<sup>5</sup>; драйвер для среды UEFI, позволяющий передавать диагностические сообщения на самых ранних этапах работы эмулятора; а также драйвер для

---

<sup>4</sup>Энциклопедия «Касперского» — UEFI

<sup>5</sup><https://www.openeuler.org/en/other/projects/stratovirt/>

операционной системы Windows, обеспечивающий возможность передачи диагностической информации от приложений, сервисов и других драйверов, работающих внутри гостевой системы.

Таким образом, постановка задачи заключается в разработке данных трёх компонентов. Для достижения поставленной цели требуется выполнить следующие этапы:

1. Изучить устройство и архитектуру эмулятора stratovirt.
2. Разработать виртуальное устройство на стороне эмулятора.
3. Изучение устройства и архитектуру UEFI.
4. Разработать драйвер для UEFI.
5. Разобраться в коде некоторых драйверов для ОС Windows и изучить API, предоставляемые операционной системой для разработки драйверов.
6. Реализовать драйвер для ОС Windows.
7. Сравнить реализованный канал для передачи данных с другими существующими.

Методологическую основу работы составляют: анализ технической документации и литературы о разработке драйверов для UEFI и Windows, материалы по lock-free алгоритмам и методы отладки, применяемые в среде виртуализации.

Постановка задачи определяет структуру дипломной работы, которая включает введение, четыре главы и заключение.

Введение задаёт цели и задачи работы: в этой части определяются объект и предмет исследования и раскрывается значимость.

В первой главе рассматриваются устройство и архитектура виртуальной машины stratovirt.

Вторая глава посвящена архитектуре UEFI и принципам разработки драйверов В среде UEFI.

В третьей главе описываются подходы к разработке драйверов в операционной системе Windows.

Четвёртая глава посвящена описанию реализации механизма передачи диагностических данных.

Заключение содержит обобщение результатов проведённого исследования, сравнение с другими механизмами, а также описание возможных сценариев использования разработанного решения.

## 2 Архитектура виртуальной машины Stratovirt

В данной главе рассматривается архитектура эмулятора Stratovirt, а именно механизмы, используемые при разработке виртуального устройства сбора диагностических данных LogDev.

### 2.1 Stratovirt в экосистеме виртуализации

Stratovirt — VMM (Virtual Machine Manager) экосистемы openEuler, ориентированный на облачные и контейнерные сценарии [1].

С точки зрения разработчика Stratovirt удобен тем, что он архитектурно схож с QEMU и имеет аналогичные параметры командной строки.

В предлагаемом механизме Stratovirt выступает в качестве хоста — приёмщика диагностических сообщений. Основой механизма является виртуальное устройство LogDev, которое принимает данные из прошивки UEFI и из гостевой ОС Windows, а затем выводит их в единый файл с диагностическими сообщениями.

### 2.2 Архитектура Stratovirt

Как и QEMU, Stratovirt запускает виртуальную машину как отдельный процесс операционной системы. Внутри процесса выделяются:

1. **главный поток** — цикл обработки событий (таймеры, сигналы, запросы QMP);
2. **потoki vCPU** — по одному на виртуальный процессор; обрабатывают выходы в гипервизор (VM-exit), а также обращения к памяти;
3. **I/O потоки — iothreads** — отдельные потоки, которые могут создавать устройства ввода-вывода для снижения нагрузки на главный цикл.

При работе с KVM большая часть инструкций гостевой ОС выполняется на физическом процессоре. Код эмулятора выполняется, например, при обращении к MMIO регистрам устройства. Этот обработчик выполняется в процессе Stratovirt-а и может инициировать дальнейшие действия, такие

как копирование сообщения из RAM гостевой системы, уведомление потока-читателя и т.д.

Опциональный параметр `iothread` виртуального устройства `LogDev` позволяет вынести обработку чтения из буфера в отдельный поток, что согласуется с принятой в `Stratovirt` архитектурой [2] и уменьшает влияние на главного потока.

## 2.3 Модель устройств

В `Stratovirt` соблюдена иерархия устройств, которые подключаются к шинам (`PCI`, `SysBus` и др.). Устройства регистрируются и инициализируются при старте эмулятора.

Важную роль в работе с устройствами играет подсистема `AddressSpace` и регионы памяти (`MemoryRegion`):

- регион типа `ram` отображает физическую память гостевой ОС;
- регион типа `io` позволяет установить обработчики чтения/записи конкретного устройства с диапазоном гостевых физических адресов;
- регион типа `Container` позволяет объединить несколько регионов памяти в один логический регион;
- регион типа `RomDevice` отображает физическую память устройства в режиме только для чтения;
- регион типа `RamDevice` отображает физическую память устройства, не входящую в основную RAM гостевой ОС (например, `PCI BAR`);
- регион типа `Alias` позволяет создать псевдоним для существующего региона памяти.

Устройство `LogDev` реализовано как устройство, присоединённое к шине `SysBus`: оно занимает фиксированный диапазон в карте памяти. Такое решение упрощает доступ к устройству из `UEFI`. В предлагаемой реализации регистры устройства `LogDev` располагаются с адреса `0x0903_0000` (в гостевой физической памяти) и занимают 48 (`0x30`) байт.

## 2.4 Добавление LogDev через параметры командной строки

Устройство добавляется в конфигурацию машины так же, как и другие виртуальные устройства:

```
1 -device logdevice ,id=<logdev_id>[,flush_logs=<true|false>]  
2 [,iothread=<iothread_id>]
```

Ниже представлено описание параметров командной строки для добавления устройства LogDev:

- `id` — уникальный идентификатор экземпляра устройства;
- `flush_logs` — поведение при переполнении кольцевого буфера в асинхронном режиме (`true`: заблокировать запись до освобождения места; `false`: пропустить сообщение, не блокируя запись);
- `iothread` — необязательный идентификатор потока I/O; при отсутствии обработка выполняется в главном цикле.

## 2.5 Регистровый интерфейс LogDev (MMIO)

Гостевая ОС взаимодействует с LogDev через MMIO. В таблице 1 перечислены регистры, поддерживаемые устройством.

**Синхронный режим** (используется на ранних стадиях UEFI — SEC и PEI): гость записывает в `sync_wr` последовательность байт размером 1, 2, 4 или 8 байт; эмулятор накапливает их во внутреннем буфере до символа окончания строки и печатает накопленное сообщение в лог эмулятора, наряду с другими сообщениями.

**Асинхронный режим** (после выделения буфера в RAM гостевой ОС): гость записывает физический адрес структуры кольцевого буфера в регистр `bufptr`. Stratovirt получает, а затем транслирует этот гостевой адрес в виртуальный адрес хостовой ОС и читает заголовок буфера (`read_pos`, `write_pos`, флаг переполнения) напрямую из памяти гостевой ОС без дополнительного копирования всего кольца на хост. После записи сообщения гость записывает магическое значение в `kick`, таким образом сигнализируя через `ioeventfd`

Таблица 1: Регистры ММЮ устройства LogDev

| Смещ. | Имя               | Доступ        | Назначение                                                                                            |
|-------|-------------------|---------------|-------------------------------------------------------------------------------------------------------|
| 0x00  | SYNC_WR           | запись        | Побайтовая передача сообщения в синхронном режиме                                                     |
| 0x08  | BUFPTR            | чтение/запись | Гостевой физический адрес кольцевого буфера                                                           |
| 0x10  | KICK              | запись        | Уведомление эмулятора о наличии данных в буфере                                                       |
| 0x18  | OVERFLOW_BEHAVIOR | чтение        | Текущее значение параметра <code>flush_logs</code> — поведение при переполнении буфера                |
| 0x20  | FLUSH             | запись        | Принудительное чтение данных из буфера: управление ОС возвращается после чтения всех данных из буфера |
| 0x28  | BUFSIZE           | чтение        | Размер полезной области буфера                                                                        |

поток-читателю, что данные готовы к чтению, и запускает обработчик чтения данных из буфера.

## 2.6 ACPI-описание для гостевых систем

Для того, чтобы операционная система могла обнаружить устройство, Stratovirt формирует запись в таблицы ACPI, добавляя в неё узел `LOGD` с идентификатором `_HID = STVRT_LOG` и ресурсом `_CRS` типа `Memory32Fixed`, описывающим ММЮ-диапазон регистров устройства. В частности, при помощи данной записи, драйвер Windows может получить информацию о буфере и обнаружить устройство.

## 2.7 Обработка диагностического сообщения в эмуляторе

На стороне Stratovirt обработка сообщения от гостевой ОС к следующей последовательности событий:

1. гость (UEFI или гостевая ОС) формирует сообщение;
2. в синхронном режиме — потоковая запись в `sync_wr`; в асинхронном — запись в кольцевой буфер в RAM и сигнализирование об окончании записи через `kick` или `flush` (в зависимости от заданного в командной строке поведения при переполнении буфера);
3. виртуальное устройство читает сообщение из буфера;
4. после чтения сообщение выводится через общую систему логирования Stratovirt (вместе с сообщениями самого эмулятора), что позволяет объединить их с сообщениями UEFI и Windows и понять более полно картину работы системы;
5. при переполнении асинхронного буфера фиксируется отдельная запись в лог эмулятора: `Log ring buffer is overflowed!`.

В случае, если был задан поток `iothread`, то ожидание сигнала от гостевой ОС через `kick_eventfd` выполняется в этом потоке, не нагружая главный цикл.

## 2.8 Сравнение с альтернативными каналами

Для обмена сообщениями между гостевой ОС и эмулятором часто используют:

- **virtio-serial/virtio-console** — нет возможности использовать на ранних стадиях UEFI, нет возможности писать в лог из нескольких потоков, дополнительные накладные расходы на обработку сообщения;
- **debugcon** — только побайтовый вывод, нет асинхронного режима, `VM-exit` при выводе каждого байта.

## 2.9 Выводы по главе

Для задачи передачи диагностических сообщений в гостевой ОС был выбран канал, основанный на разделяемом кольцевом буфере и ММЮ-регистре уведомления: копирование выполняется один раз в общий буфер, из которого по сигналу от гостевой ОС эмулятор читает сообщение.

1. Stratovirt — менеджер виртуальных машин на базе KVM и QEMU-подобной модели устройств, используется в качестве эмулятора;
2. LogDev — виртуальное устройство, присоединённое к шине SysBus, имеющее фиксированную область ММЮ-регистров и ACPI-запись, что позволяет обнаружить устройство в Windows;
3. Синхронный и асинхронные режимы работы позволяют обращаться к устройству на всех стадиях работы виртуальной машины, в том числе и раннюю загрузку прошивки UEFI;
4. Устройство кольцевого буфера рассматривается в разделе 5.

### 3 Драйверы в среде UEFI

Данная глава посвящена архитектуре UEFI и разработке драйверов для неё в среде EDK2. Драйвер для виртуального устройства в эмуляторе Stratovirt необходим для выделения памяти для кольцевого буфера, а также для поддержки записи сообщений в буфер из модулей прошивки.

UEFI (Unified Extensible Firmware Interface) — интерфейс между операционной системой и микропрограммами, управляющими низкоуровневыми функциями оборудования. Стандарт UEFI выделяет несколько основных стадий. Для разработки виртуального устройства для логгирования наиболее важны нюансы работы с памятью на ранних этапах прошивки. Например, на стадии SEC и PEI оперативная память ещё недоступна: вместо неё используется кэш процессора. Это ограничивает нас в работе с общей памятью в традиционном понимании, поэтому было разработано два режима работы с логирующим устройством:

- **синхронный ММЮ** — побайтовая запись в регистр устройства `sync_wr`;
- **запись через буфер в оперативной памяти гостевой ОС** — кольцевой буфер с уведомлением о записи через регистр виртуального устройства `kick`. Доступен после инициализации памяти и передачи адреса буфера через регистр виртуального устройства `bufptr`.

#### 3.1 Обзор фаз Platform Initialization

Спецификация UEFI PI определяет стадии работы прошивки [3]:

- **SEC** — Security Phase;
- **PEI** — Pre-EFI Initialization;
- **DXE** — Driver Execution Environment;
- **TSL** — Transient System Load;
- **RT** — Runtime.

При переходе от одной фазы к другой изменяется:

- доступная память;
- таблицы сервисов;
- набор загружаемых модулей.

Так, на ранних этапах (SEC и PEI) контроллер памяти ещё не инициализирован, поэтому работа напрямую с ОЗУ недоступна. Также на разных стадиях доступны разные сервисы — API, которые UEFI предоставляет разработчику модуля. Каждая фаза также характеризуется набором загружаемых модулей — исполняемых компонентов прошивки.

Сводное сопоставление приведено в таблице 2.

Таблица 2: Фазы PI/UEFI и канал диагностики в разрабатываемом решении

| Фаза | Память                      | Работа с устройством          | Особенности разработки драйвера устройства                                                |
|------|-----------------------------|-------------------------------|-------------------------------------------------------------------------------------------|
| SEC  | Кэш процессора              | Синхронное через MMIO SYNC_WR | Для записи доступен только стек                                                           |
| PEI  | Кэш процессора + ОЗУ        | Синхронное через MMIO SYNC_WR | На данном этапе происходит инициализация RAM — после неё можно выделить память для буфера |
| DXE  | Полная                      | Асинхронная через буфер       | —                                                                                         |
| TSL  | Полная                      | Асинхронная через буфер       | —                                                                                         |
| RT   | Полная + виртуальные адреса | Асинхронная через буфер       | Подписка на <code>SetVirtualAddressMap</code> для сохранения виртуальных адресов буфера   |

### 3.2 Фаза SEC (Security)

**Назначение.** SEC — первая фаза прошивки после сброса CPU. Её основными задачами являются: обработка аппаратных исключений, инициализация временной памяти через кэш, передача управления следующей стадии — PEI [3].

**Ограничения.** На данном этапе нет таблиц с сервисами. Нельзя вызывать `AllocatePool`, `AllocateRuntimePool` для выделения памяти под буфер, так как ОЗУ ещё не инициализирована и используется временная память через кэш процессора.

**Особенности разработки драйвера.** На данной стадии происходят самые ранние сбои, которые трудно отладить без специальных инструментов. Из-за ограничений на работу с памятью для обмена сообщениями с устройством на этой стадии используется синхронная запись в специальный регистр устройства `sync_wr`. Драйвер записывает сообщение в регистр порциями по 1, 2, 4 или 8 байт, а эмулятор накапливает эти сообщения, пока не встретит символ окончания строки `\0` [4].

### 3.3 Фаза PEI (Pre-EFI Initialization)

**Назначение.** Стадия PEI Core последовательно запускает PEIM (PEI modules), публикующие PPI (PEI-to-PEI Interface), необходимые для предоставления интерфейса для обращений от других модулей. На данной стадии также происходит инициализация физической памяти, что позволяет на последующих этапах отойти от использования синхронного регистра `sync_wr` для записи сообщений.

#### 3.3.1 PEI до инициализации ОЗУ (pre-memory)

Как и на стадии SEC, здесь доступна лишь временная память через кэш процессора, поэтому ограничения работы с виртуальным устройством остаются теми же:

- работа возможна лишь через **синхронный ММЮ**;
- выделение страниц под кольцевой буфер по-прежнему невозможно.

#### 3.3.2 PEI после инициализации ОЗУ (post-memory)

После выставления PPI `MemoryDiscovered` PEIM получают доступ к методу `PeiServicesAllocatePages`, позволяющему выделить память в ОЗУ. Поэтому на

данном этапе производится резервирование памяти, которая представляет собой:

1. непрерывный диапазон страниц под заголовок кольцевого буфера и массив данных;
2. тип памяти — `runtime`, благодаря чему память может быть доступна после вызова `ExitBootServices` во время работы ОС (таких как Windows или Linux);

Также после выделения памяти адрес сообщается эмулятору через регистр `VFPT` виртуального устройства. На дальнейших этапах модули могут получить адрес буфера через этот регистр.

### 3.4 Фаза DXE (Driver Execution Environment)

**Назначение.** На данной стадии происходит загрузка драйверов устройств (например, USB, графических, сетевых и т.д.), предоставляется набор сервисов и протоколов (например, GOP (Graphics Output Protocol) для графики) и подготавливается среда для загрузчика ОС [5, 6].

**Особенности разработки драйвера.** DXE-драйвер (`DXE_DRIVER`) виртуального устройства выполняет:

1. **получение адреса буфера** — чтение регистра `VFPT` с адресом буфера;
2. **обработку переполнения** — чтение регистра `OVERFLOW_BEHAVIOR`, которое согласовано с параметром `flush_logs` строки запуска эмулятора, а также чтение регистра `VFBSIZE` с размером буфера;
3. **регистрацию событий** — подписка на событие смены виртуальных адресов `SetVirtualAddressMap` для поддержки работы на последующих стадиях;
4. **публикацию протокола записи** — публикация интерфейса для обращения к устройству `LogDev` из других модулей.

**Обращение к устройству.** Как правило, модули обращаются к виртуальному устройству не напрямую, а через специальную реализацию `DebugLib`,

которая обрабатывает сообщения и в конечном счёте обращается к драйверу для отправки сообщения. Драйвер же, в свою очередь, копирует сообщение в кольцевой буфер (раздел 5) и по окончании записи оповещает устройство при помощи регистра `kick`.

### 3.5 Фаза TSL (Transient System Load)

**Назначение.** Данная стадия — переходный этап между прошивкой и ОС. TSL начинается, когда управление передаётся загрузчику операционной системы. На этом этапе всё ещё доступны Boot Services (до тех пор, пока не будет вызвана функция `ExitBootServices`).

На этой фазе также могут печататься важные для отладки сообщения. Для драйвера `LogDev` никаких дополнительных действий выполнять на этой фазе не нужно: буфер уже был зарезервирован, а его адрес можно получить из регистра `BUFPTR`.

### 3.6 Фаза Runtime

**ExitBootServices.** Загрузчик ОС в какой-то момент вызывает `ExitBootServices`. Все зарегистрированные обработчики события выхода из Boot Services выполняются до возврата из `ExitBootServices`. Как правило, в них происходит освобождение ресурсов, используемых модулями на предыдущих этапах. Как только завершается вызов `ExitBootServices`, начинается фаза Runtime.

**SetVirtualAddressMap.** Когда ОС переходит на виртуальные адреса, вызывается runtime service `SetVirtualAddressMap`, при исполнении которого вызываются ранее установленные обработчики. Обработчик `EVT_VIRTUAL_ADDRESS_CHANGE` обязан оттранслировать указатели runtime-кода UEFI в виртуальные согласно новой таблице [3]. Так поступает и обработчик `LogDev`, который обновляет свой адрес буфера на виртуальный. Без этого шага драйвер обращался бы к неверным адресам. Однако, эмулятор может работать с физической памятью напрямую, поэтому сообщать ему об изменении адреса буфера нет необходимости.

### 3.7 Перезагрузка

Стоит также упомянуть, что происходит при перезагрузке операционной системы. После деинициализации ОС вызывает UEFI Runtime Service `ResetSystem`, который никогда не возвращает управление. В обработчике данной функции также могут печататься немаловажные отладочные сообщения. Резонно в эмуляторе добавить слой безопасности и запретить перезапись адреса буфера в `virt`. В таком случае, однако, нужно оставить возможность перезаписи адреса буфера, т.к. при повторной работе UEFI будет выделен новый буфер с иным адресом. Как правило, во многих эмуляторах (в том числе в `Stratovirt`) есть возможность подписаться на перезагрузку гостевой операционной системы и сбросить адрес буфера.

### 3.8 Отладочная библиотека

Помимо драйвера была разработана специальная реализация библиотеки `DebugLib`. В стандартной сборке EDK2 макросы `DEBUG(...)` и `ASSERT(...)` выводят сообщения, как правило, в последовательный (`serial`) порт. Разработанная реализация `DebugLib` перенаправляет эти вызовы в виртуальное устройство `LogDev` через UEFI-драйвер. Отладочные сообщения от модулей прошивки, уже использующих `DebugLib`, направляются в `LogDev` без необходимости изменять код существующих модулей.

### 3.9 Выводы по главе

Драйвер UEFI реализован при помощи экосистемы EDK2 [5]: он отвечает за работу с устройством на ранних этапах загрузки (пока память ещё не инициализирована), за выделение памяти под буфер и осуществление работы с ним в асинхронном режиме. Кастомная `DebugLib`, описанная в начале главы, обеспечивает доступ модулей прошивки к `LogDev` без их модификации.

Итого:

1. ранние стадии UEFI (SEC и PEI) допускают только MMIO через регистр `sync_wr`; после появления RAM возможно выделение страниц под буфер и

сообщение адреса устройству через регистр `WUPTR`;

2. `SetVirtualAddressMap` обеспечивает возможность логирования из `Runtime Services`.

## 4 Драйверы в операционной системе Windows

В данной главе рассматривается архитектура драйверов Windows и способы обращения пользовательских приложений и драйверов режима ядра к общему кольцевому буферу.

### 4.1 Задача драйвера устройства логирования

В гостевой ОС Windows диагностические сообщения могут быть сгенерированы:

- приложения и сервисы пользовательского режима;
- другие драйверы режима ядра.

В рамках данной задачи подразумевается, что диагностические сообщения могут отправлять "доверенные" приложения, сервисы и драйверы, необходимые для бесшовной работы виртуальной машины.

Драйвер играет роль единой точки доступа к кольцевому буферу: он предоставляет интерфейс для записи сообщений в буфер, совершая проверки его целостности, и уведомляет эмулятор через ММIO-регистр киск 2.

### 4.2 Схема взаимодействия с драйвером

Модель Windows разделяет выполнение на пользовательский режим и режим ядра. Запросы к устройствам оформляются как IRP (I/O Request Packet), которые проходят стек драйверов, начиная с I/O Manager [7].

Схема взаимодействия с драйвером:

1. PnP Manager обнаруживает устройство через ACPI-таблицы;
2. функциональный драйвер создаёт Device Object (структура ядра Windows, предоставляющая интерфейс для I/O взаимодействия с устройством);
3. пользовательский код открывает символьную ссылку на устройство (`\\.\LogDev`); и вызывает `DeviceIoControl`;
4. драйвер обрабатывает IOCTL и обращается к разделяемой памяти.

### 4.3 Модель разрабатываемого драйвера

В рамках данной задачи драйвер был разработан по модели WDM (Windows Driver Model) в силу меньшего количества накладных расходов. Точкой входа в драйвер в данной модели является функция `DriverEntry`, инициализирующая структуру `DRIVER_OBJECT` и регистрирующая обработчики для различных типов запросов. `DriverUnload` является обработчиком для освобождения ресурсов, выделенных драйвером.

В `DriverEntry` регистрируются обработчики для различных типов запросов:

- `MajorFunction[IRP_MJ_CREATE]` — открытие устройства (вызывается при `CreateFile` или `OpenFile` в пользовательском режиме);
- `MajorFunction[IRP_MJ_CLOSE]` — закрытие (вызывается при `CloseHandle` в пользовательском режиме);
- `MajorFunction[IRP_MJ_DEVICE_CONTROL]` — обработка IOCTL (вызывается при `DeviceIoControl` с определенным кодом в пользовательском режиме);
- `MinorFunction[IRP_MN_START_DEVICE]` — запрос, который PnP менеджер отправляет драйверу для запуска устройства;
- `AddDevice` — данная функция вызывается PnP менеджером, когда устройство обнаруживается. Основной задачей данной функции является создание `Device Object` и прикрепление его к стеку устройств.

### 4.4 Обнаружение буфера через ACPI

Для большей универсальности, адрес и размер ММЮ регистров передается через ACPI-таблицы. Далее, драйвер из регистров получает физический адрес и размер кольцевого буфера, выделенного в UEFI.

Напомним, что эмулятор добавляет в таблицы ACPI устройство со следующими параметрами [4]:

- имя ACPI-узла: `LOGD`;

- `_HID`: `STVRT_LOG`;
- `_CRS`: дескриптор `Memory32Fixed` для ММЮ-региона устройства `LogDev` (база и длина согласованы с картой памяти эмулятора).

Драйвер в обработчике `IRP_MN_START_DEVICE` получает структуру `IO_STACK_LOCATION` и извлекает из неё ресурсы устройства (дескриптор `CM_RESOURCE_LIST`). Из него извлекается диапазон ММЮ для регистров (таких как `BUFPTR`, `KICK` и др.). Далее драйвер получает физический адрес и размер кольцевого буфера и отображает физические страницы в виртуальное адресное пространство при помощи функций `MmMapIoSpace` / `MmGetPhysicalAddress` (в зависимости от типа памяти — ММЮ регистры или RAM) и сохраняет указатель на структуру заголовка буфера.

#### 4.5 Интерфейс IOCTL для пользовательского режима

Для приложений и служб `user-mode` предусмотрен IOCTL, принимающий готовое сообщение (NUL-терминированная строка или структура с полем длины). Обработчик в `IRP_MJ_DEVICE_CONTROL`:

1. проверяет входной буфер (`IoGetCurrentIrpStackLocation`);
2. валидирует размер (не превышает максимального сообщения кольца);
3. вызывает общую функцию записи в lock-free буфер (та же, что в UEFI);
4. при успехе записывает в ММЮ `KICK` значение `0xDEADBA6A` для пробуждения потока `Stratovirt`.

Коды IOCTL задаются через `CTL_CODE` с уникальным `DeviceType` и функциями `METHOD_BUFFERED` (копирование буфера ядром) либо `METHOD_NEITHER` при жёстком контроле `Probe` в драйвере [7].

#### 4.6 Интерфейс для драйверов режима ядра

Второй IOCTL (или экспорт через `IoCreateDevice` + интерфейс `GUID`) возвращает указатель на функцию записи в буфер:

```
1 typedef NTSTATUS (*LOGDEV_WRITE_FN)(  
2     PVOID Message ,  
3     ULONG MessageSize  
4 );
```

Другой драйвер, получив указатель при инициализации, может логировать без прохождения user-kernel transition и без повторного отображения памяти. Это снижает задержку при высокой частоте событий от сетевого или дискового стека.

Проверка подписи вызывающего модуля (опционально) выполняется через сравнение уровня IRQL и принадлежности к загруженным драйверам; в учебной реализации достаточно ограничить ИОСТЛ привилегированным открытием устройства.

#### 4.7 Взаимодействие с Stratovirt после записи

Последовательность для асинхронного режима совпадает с UEFI:

1. `reserve` — резервирование места в кольце (атомарные операции над `ShwPos`, см. раздел 5);
2. копирование тела сообщения;
3. `commit` — обновление `WritePos` при нулевом счётчике активных писателей;
4. запись в ММЮ кик или явный `flush`.

Поток `iothread` на хосте считывает сообщения и печатает их в общий лог Stratovirt с префиксом `GUEST(ptr mode)`, что позволяет объединить их с логами эмулятора и с сообщениями UEFI, как указано во введении.

#### 4.8 Сравнение путей доступа

#### 4.9 Выводы по главе

1. Драйвер Windows связывает user-mode и kernel-mode клиентов с тем же кольцевым буфером, что инициализирует UEFI.

Таблица 3: Способы записи диагностики в гостевой Windows

| Клиент              | API                          | Особенности                                             |
|---------------------|------------------------------|---------------------------------------------------------|
| Приложение / служба | CreateFile + IOCTL           | Проверка буфера в ядре, стандартная модель безопасности |
| Другой драйвер      | указатель<br>LOGDEV_WRITE_FN | Минимальная задержка, без syscall                       |
| Прошивка (до ОС)    | DebugLib / MMIO              | Описано в главе 3                                       |

2. ACPI `stvirt_log` / `logd` обеспечивает обнаружение MMIO-устройства и согласованность с Stratovirt.
3. Два IOCTL (запись сообщения и получение указателя на функцию записи) покрывают основные сценарии диагностики в госте.
4. Детали lock-free алгоритма и структуры буфера приведены в разделе 5.

## 5 Реализация механизма сбора диагностической информации

Разработанный механизм сбора диагностической информации включает три основных компонента:

1. Виртуальное устройства на стороне гипервизора Stratovirt.
2. Драйвер для среды UEFI.
3. Драйвер для операционной системы Windows.

Каналом взаимодействия между данными компонентами служит кольцевой буфер, расположенный в общей памяти, разделяемой между гостевой ОС и гипервизором.

Виртуальное устройство эмулируется в гипервизоре Stratovirt и выступает в роли единой точки сбора диагностической информации. Таким образом, основными функциями устройства являются:

1. Приём диагностических сообщений от драйверов гостевых систем.
2. Сериализация сообщений и запись их в файл.
3. Хранение метаинформации о буфере: его размере и типе поведения при переполнении.
4. Предоставление доступа к метаинформации посредством регистров виртуального устройства.

Драйвер для среды UEFI выполняет следующие задачи:

1. Выделение буфера в гостевой памяти типа runtime: доступной после окончания работы UEFI (после вызова `ExitBootServices`) и во время работы операционной системы.
2. Предоставление интерфейса для записи диагностических сообщения в буфер.
3. Регистрация обработчиков событий `ExitBootServices` и `SetVirtualAddressMap` для обеспечения работы драйвера с `EfiRuntimeService`.

Для упрощения интеграции с существующими UEFI модулями была разработана собственная версия `DebugLib`, переопределяющая стандартные функции логирования. Эти функции обращаются к вышеупомянутому драйверу для передачи сообщений.

Для обеспечения взаимодействия компонентов гостевой системы с буфером реализован драйвер, позволяющий:

1. User-space приложениям и сервисам взаимодействовать с буфером.
2. Kernel-space драйверам взаимодействовать с буфером.

Общая архитектура механизма представлена на рисунке 1.



Рис. 1: Архитектура механизма обмена диагностической информацией

Виртуальное устройство может быть добавлено в устройство при помощи флага `-logdevice`, который принимает следующие параметры: `id`, `flush_logs`, `iothread`:

1. `id` - идентификатор виртуального устройства, позволяющий однозначно определить его в системе.
2. `flush_logs` - параметр логического типа, который регламентирует поведение при переполнении буфера. Если параметр установлен в `true`, запись сообщения будет ожидать освобождение места в буфере. Если `false` - сообщение, не поместившееся в буфер будет пропущено.
3. `iothread` - идентификатор контекста, в котором работает поток-читатель. Параметр может быть пропущен — в таком случае поток-читатель будет работать в главном контексте (`main loop`).

Виртуальное устройство взаимодействует с гостевыми операционными

системами через регистров, доступ к которым осуществляется посредством ММЮ (memory-mapped I/O). В частности, среди прочих существует регистр, позволяющий получить информацию о поведении при переполнении буфера.

Как уже отмечалось, выделение памяти буфера выполняется драйвером UEFI. Однако на ранних стадиях работы прошивки (SEC и PEI) эта операция невозможна, т.к. память ещё не инициализирована. По этой причине, на этих этапах требуется альтернативный способ передачи диагностических данных в виртуальное устройство, поэтому в него был добавлен специальный регистр, позволяющий записываются данные последовательно — по 8, 4, 2 байта или по одному в зависимости от размера сообщения. Виртуальное устройство накапливает поступающие байты до тех пор, пока не встретит символ окончания строки '\0', после чего сформированное сообщение записывается в выходной файл.

После инициализации памяти (на этапе PEIM) появляется возможность выделить буфер. Дальнейшая передача сообщений происходит уже через этот буфер.

Общая память организована в виде неблокирующего (lock-free) кольцевого буфера с несколькими писателями и одним читателем (MPSC — Multiple Producer Single Consumer). В начале буфера располагается заголовок, содержащий указатели текущих позиций читателя и писателя, флаг переполнения, а также вспомогательную структуру SHADOW\_POS ("мнимая позиция"). Данная структура включает два поля типа UINT32: счётчик активных писателей, выполняющих копирование сообщений, и позицию резервирования места для новых сообщений.

Структура буфера представлена ниже:

```
1  typedef union {  
2      struct {  
3          uint32_t ReservedPos;  
4          uint32_t WriterCnt;  
5      } S;  
6      uint64_t U;  
7  } SHADOW_W_POS;
```

```
8
9     typedef struct {
10         uint32_t ReadPos;
11         uint32_t WritePos;
12         uint64_t IsOverflow;
13         SHADOW_W_POS ShWPos;
14         char Buffer[0];
15     } LOCKLESS_RING_BUFFER;
```

Процедура записи в буфер выполняется в три этапа: резервирование места для сообщения (reserve), копирование в забронированную область (copy) и оповещение об окончании копирования (commit).

Сначала каждый поток пытается зарезервировать место для своего сообщения. Для этого каждый поток загружает текущее значение SHADOW\_POS и с помощью атомарной операции compare\_exchange пытается обновить его в заголовке буфера, увеличивая счётчик писателей и смещая позицию записи на размер сообщения. В случае конфликта (если compare\_exchange вернула значение false) операция повторяется.

Таким образом, гарантируется, что никакой другой поток не сможет зарезервировать себе то же самое место: каждый поток копирует своё сообщение в уникальную область буфера.

Псевдокод данного этапа представлен ниже:

```
1     uint32_t LrbReserve (
2         LOCKLESS_RING_BUFFER *Rb,
3         const uint32_t MsgSize,
4         SHADOW_W_POS *Curr
5     ) {
6         for (;;) {
7             Curr->U = Rb->ShWPos.U;
8             SHADOW_W_POS NextShadowPos = {
9                 .S.ReservedPos = Curr->S.ReservedPos + MsgSize,
10                .S.WritePos = Curr->S.WriterCnt + 1
11            };
12
13            if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
14                // SHADOW_W_POS была обновлена в буфере
```

```
15         // Таким образом, гарантируется, что ни один другой поток
16         // не будет копировать своё сообщение в занятую память.
17         return Curr->S.ReservedPos;
18     }
19 }
20 }
```

После того, как поток скопировал сообщение, он сообщает об окончании операции. Поток атомарно уменьшает счётчик активных писателей в SHADOW\_POS, используя аналогичный резервированию механизм синхронизации. Как только количество писателей в SHADOW\_POS становится равным нулю, обновляется глобальный указатель писателя в заголовке буфера. Таким образом, указатель писателя обновляется после завершения всех операций записи, что гарантирует целостность данных в буфере и исключает чтение неполных записей на читающей стороне.

Псевдокод алгоритма представлен ниже:

```
1  uint32_t LrbCommit (
2      LOCKLESS_RING_BUFFER *Rb,
3      SHADOW_W_POS *Curr
4  ) {
5      for (;;) {
6          SHADOW_W_POS NextShadowPos = {
7              .S.ReservedPos = Curr->S.ReservedPos,
8              .S.WritePos = Curr->S.WriterCnt - 1
9          };
10
11          if (NextShadowPos.S.WriterCnt == 0) {
12              // все сообщения до текущего закончили копирование
13              Rb->WritePos = Curr->S.ReservedPos;
14          }
15          if (atomic_compare_exchange(&Rb->ShWPos, &Curr->U, Next.U)) {
16              return;
17          }
18      }
19 }
```

После завершения записи драйвер уведомляет виртуальное устройство через отдельный регистр о том, что глобальная позиция писателя изменилась

и сообщения готовы к чтению.

Функции записи также должны проводить некоторые проверки буфера, чтобы предотвратить переполнение и запись в повреждённый буфер. Наивный алгоритм проверки мог бы выглядеть следующим образом:

```
1  uint32_t LrbReserve (  
2      LOCKLESS_RING_BUFFER *Rb,  
3      const uint32_t MsgSize,  
4      SHADOW_W_POS *Curr  
5  ) {  
6      for (;;) {  
7          Curr->U = Rb->ShWPos.U;  
8          uint32_t ReadPos = Rb->ReadPos;  
9          uint32_t BusySize = Curr->S.ReservedPos - ReadPos;  
10         if (BusySize > BUFFER_SIZE) {  
11             // Буфер сломан:  
12             // размер занятой области не может быть больше всего буфера.  
13             return -1;  
14         }  
15         if (BusySize + Size > BUFFER_SIZE) {  
16             // Сообщение не поместилось.  
17             // В зависимости от выбранного поведения, сообщение либо  
18             // пропускается, либо ожидает завершения записи других.  
19         }  
20  
21         ...  
22     }  
23 }
```

Однако при таком подходе возникает риск ложно принять буфер за повреждённый, если между операциями чтения структуры SHADOW\_POS и проверкой целостности буфера другой поток успеет зарезервировать место в буфере, скопировать сообщение и выполнить операцию commit. Рассмотрим это на примере:

Пусть изначально:

1. Позиция читателя ReadPos = 100.
2. Аисателя WritePos = 200.
3. Количество активных писателей ShWPos.WriterCnt = 1.

При описанном ниже планировании потоков возникает указанная проблема.

| Поток-писатель №1                                                                                                                                                                                                                                                                       | Поток-писатель №2                                                                                                                                                         | Поток-читатель                                      |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| (1) Операция резервирования:<br>$\text{Curr} \rightarrow \text{U} = \text{Rb} \rightarrow \text{ShWPos.U}$<br>$(\Rightarrow \text{Curr} \rightarrow \text{S.ReservedPos} = 200)$<br>// Поток не увеличил $\text{ShWPos.WriterCnt}$<br>// Другие потоки могут обновить $\text{WritePos}$ |                                                                                                                                                                           |                                                     |
|                                                                                                                                                                                                                                                                                         | (2) Операция резервирования:<br>$\text{ShWPos.U} = \text{Cur} \rightarrow \text{U}$<br>$(\Rightarrow \text{ShWPos.S.ReservedPos} = 100)$                                  |                                                     |
|                                                                                                                                                                                                                                                                                         | (3) Операция commit:<br>$\text{WritePos} = \text{ShWPos.S.ReservedPos} = 300$<br>// $\text{WritePos}$ смог измениться, т.к.<br>// поток №1 не увеличил $\text{WriterCnt}$ |                                                     |
|                                                                                                                                                                                                                                                                                         |                                                                                                                                                                           | Читает:<br>$\text{ReadPos} = \text{WritePos} = 300$ |
| Операция резервирования:<br>$\text{uint32\_t ReadPos} = \text{Rb} \rightarrow \text{ReadPos} = 300$<br>$\text{uint32\_t BusySize} = \text{Curr} \rightarrow \text{S.ReservedPos} - \text{ReadPos} =$<br>$= 200 - 300$                                                                   |                                                                                                                                                                           |                                                     |

Таблица 4: Пример планирования потоков

Таким образом, переменная  $\text{BusySize}$  переполняется, и буфер ложно считается повреждённым.

Данную проблема решается повторной загрузкой значений  $\text{Curr} \rightarrow \text{U}$  и  $\text{ReadPos}$ . Если  $\text{Curr} \rightarrow \text{U}$  не изменилось, то буфер действительно повреждён. В противном случае проверку необходимо повторить.

Псевдокод проверки представлен ниже:

```

1  uint32_t LrbReserve (
2      LOCKLESS_RING_BUFFER *Rb,
3      const uint32_t MsgSize,
4      SHADOW_W_POS *Curr
5  ) {
6      for (;;) {
7          Curr->U = Rb->ShWPos.U;
8          uint32_t ReadPos = Rb->ReadPos;
9          uint32_t BusySize = Curr->S.ReservedPos - ReadPos;
10
11          while (BusySize > BUFFER_SIZE) {

```

```
12         uint64_t CurrCpy = Curr->U;
13
14         // Принудительно пересчитать значения ReadPos и ShWPos
15         MemoryFence();
16         Curr->U = Rb->ShWPos.U;
17         ReadPos = Rb->ReadPos;
18
19         if (Curr->U == CurrCpy) {
20             return -1;
21         }
22         BusySize = Curr->S.ReservedPos - ReadPos;
23     }
24     if (BusySize + Size > BUFFER_SIZE) {
25         // Сообщение не поместилось.
26         // В зависимости от выбранного поведения, сообщение либо
27         // пропускается, либо ожидает завершения записи других.
28     }
29
30     ...
31 }
32 }
```

Алгоритм потока-читателя ничем в сущности не отличается от чтения из обычного кольцевого буфера. При получении уведомления об изменении позиции писателя через регистр виртуального устройства, поток-читатель последовательно извлекает сообщения, начиная с позиции ReadPos и продвигаясь к WritePos, пока не будут прочитаны все доступные данные.

Вышеописанный алгоритм читателя реализован как в драйвере UEFI, так и в драйвере для Windows.

В драйвере для Windows были реализованы ioctl - команды для взаимодействия с буфером:

1. Для пользовательского программ (user-space): она принимает сообщение и выполняет проверки над ним и записывает его в буфер.
2. Для программ режима ядра (kernel-space) — возвращает указатель на функцию записи в буфер.

Упомянуть про особенности драйвера Windows: получение адреса из

АСРІ

## 6 Заключение

Существуют и другие схожие решения, в частности кольцевой буфер, используемый в eBPF [8], в котором этап резервирования реализован с применением спинлоков. Такой подход обеспечивает корректность конкурентной записи, однако в условиях виртуализации и одновременной работы нескольких виртуальных машин использование блокирующих примитивов синхронизации может приводить к существенным задержкам. В случае переключения контекста внутри гостевой операционной системы удерживаемая блокировка препятствует прогрессу остальных потоков-писателей. Своё влияние на непредсказуемость задержек также оказывает планирование виртуальных процессоров на стороне гипервизора, что может существенно увеличить время удержания блокировки.

Преимуществом предложенного решения перед механизмом, используемым в Linux eBPF, является возможность отобразить кольцевой буфер непосредственно в виртуальное адресное пространство пользовательского процесса. Благодаря этому запись диагностических сообщений может выполняться без обращения к драйверу, что снижает накладные расходы на системные вызовы и переключение контекста.

Кроме того, в условиях одновременной работы нескольких виртуальных машин предложенное решение демонстрирует более предсказуемые задержки. В случае с eBPF блокирующие примитивы синхронизации (спинлоки) в сочетании с переключением контекста внутри гостевой ОС и планированием виртуальных процессоров могут приводить к более непредсказуемым задержкам.

В рамках данной работы был разработан альтернативный механизм передачи диагностических данных, основанный на неблокирующем MPSC-кольцевом буфере, размещённом в памяти гостевой операционной системы. Для синхронизации конкурентных записей используется вышеописанный lock-free алгоритм с разделением операций резервирования, копирования и фиксации записи (commit), что позволяет полностью отказаться от блокирую-

щих примитивов синхронизации. Предложенное решение обеспечивает независимый прогресс потоков и корректную работу даже при условии приостановки выполнения отдельных потоков-писателей. Реализация интегрирована в виртуализированную среду, представленную эмулятором Stratovirt, UEFI EDK2 и гостевой операционной системой Windows. Данный механизм позволяет обеспечить более предсказуемое поведение при высокой интенсивности диагностических событий.

## Список литературы

- [1] *openEuler Community*. StratoVirt. — <https://www.openeuler.org/en/other/projects/stratovirt/>. — 2024.
- [2] *openEuler Community*. StratoVirt Design. — <https://gitee.com/openeuler/stratovirt/blob/master/docs/design.md>. — 2024.
- [3] *UEFI Forum*. UEFI Specification. — <https://uefi.org/specifications>. — 2022.
- [4] *Абубакиров, В. А.* StratoVirt with LogDev device (branch as\_device\_lockless). — [https://gitee.com/abubakirovvladimir/stratovirt/tree/as\\_device\\_lockless/](https://gitee.com/abubakirovvladimir/stratovirt/tree/as_device_lockless/). — 2024.
- [5] *TianoCore*. EDK II. — <https://github.com/tianocore/edk2>. — 2024.
- [6] *TianoCore*. EDK II UEFI Driver Writer's Guide. — <https://tianocore-docs.github.io/edk2-UefiDriverWritersGuide/>. — 2020.
- [7] *Microsoft*. Windows Driver Kit documentation. — <https://learn.microsoft.com/en-us/windows-hardware/drivers/>. — 2024.
- [8] *Linux Foundation*. BPF ring buffer. — <https://docs.kernel.org/bpf/ringbuf.html>. — 2024.
- [9] *QEMU Project*. QEMU Documentation. — <https://www.qemu.org/docs/master/>. — 2024.
- [10] *UEFI Forum*. Advanced Configuration and Power Interface Specification. — <https://uefi.org/specifications>. — 2022.
- [11] *Росстандарт*. ГОСТ Р 56938-2016. Защита информации при использовании технологий виртуализации. — 2016.